

并行与分布式计算作业3

- 姓名：张昕跃
 - 学号：2023202300
 - 班级：2023级图灵实验班
-

思考 1

问题：CUDA 软件、硬件映射关系是什么样的？

答：

软件上的 thread 在硬件上对应 core 中的逻辑线程，32 个线程组成 warp 作为调度最小单位；
多个 warp 构成软件上 block 对应硬件分配的单个 SM (Multicore Processor)；（block间不同步，可以以任何顺序并行或串行执行）
多个 block 构成软件上 grid，对应硬件在 GPU（Device）上分布调度；
主机端的 kernel 调用映射到 GPU（Device）上具体的 grid 执行，可以扩展到不同数量的parallel cores中。

换句话说，以下是一一对应的。
软件：grid -> block -> threads
硬件：GPU(device) -> SM (multicore processor) -> core

思考 2

问题：Warp 是如何执行的？

答：

在 SM 内部，32 个线程为一组组成 warp 为单位发射，采用 SIMT（Single Instruction Multiple Threads）模式执行，共享warp scheduler。
每个 warp scheduler 调度器在一个 SM 上轮询可运行的 warp，将它们的指令发射到执行单元；（即同一时间只有一个warp在SM执行）
如果 warp 因访存或同步而等待，调度器会切换到其他 warp，以掩藏延迟。
虽然每个线程有自己的寄存器和线程上下文，但在同一条指令上它们是锁步前进的，只有控制流分歧时才会被分段执行。（即同一个warp内的线程执行同样的语句）

或者更简单地说，
warp内的线程执行同样语句，物理上并行执行；
warps和blocks逻辑上并行执行；
block内的线程同步操作。

思考 3

问题：什么是 warp divergence？如何解决？

答：

warp divergence 指，当 warp 内线程遇到条件分支并走向不同的执行路径时，会发生 divergence，导致每条路径被串行执行。（即同一个warp内的不同线程在做不同的事情，这些路径会被串行化从而增加性能开销）

解决方法：

- 1. 让不同的warp执行不同thread的代码，branch 间隔安排为 warp size 的整数倍

例如把 `if (threadIdx.x > 2)` 改为 `if (threadIdx.x / WARP_SIZE > 2)`

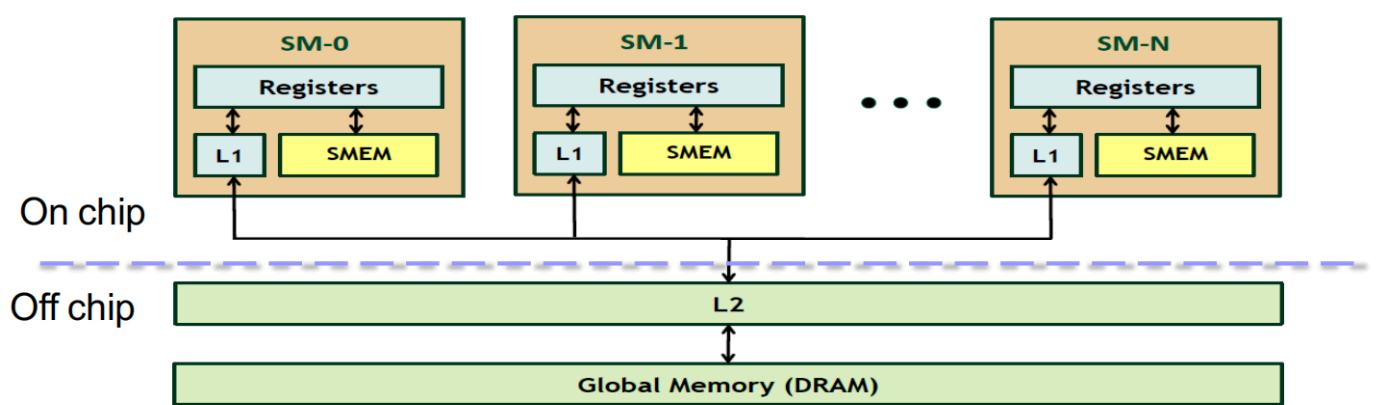
- 2. 对于不可预测的数据，用一种扁平的分布，让每个线程作用于多个数据项；
- 3. 用循环展开消除branches，增加pipeline，也可以用 `#pragma unroll` 展开；（数据依赖时还是没用）

思考 4

问题：GPU 里面的存储层级结构是什么样的？存储层次结构有什么区别？

答：

参照这张图：



40

- 1. 寄存器（每线程私有、最快） / Local Memory（线程私有，寄存器分配，bytes 存在 global 中）；
- 2. Shared Memory / L1 缓存（同一 block 共享，访问延迟低，store & load）；
- 3. L2 缓存（跨 GPU 共享，缓存全局内存数据）；
- 4. Global Memory（容量大，高延迟，not cached）；
- 5. Constant Memory（read only，cached）；

各层在容量、访问延迟、可见范围和访问模式优化上不同，除了上面括号里给出的区别，还有：

寄存器/Local：每线程私有，寄存器在片上最快但容量极小；溢出到 local memory（其实在global里）就变成高延迟访问。

Shared memory：每个 SM 上的片上 SRAM，同一 block 共享、低延迟高带宽，有 bank 冲突。

Global/Constant：位于显存、全设备可见、容量最大但延迟最高，全局访问要尽量合并。

思考 5

问题：为什么要设计动态并行，可以用在哪？

答：

dynamic parallel 可以减少 kernel launch 的次数，它允许 device kernel 再次启动子 kernel，避免频繁返回 cpu 调度，降低 CPU 参与延迟与通信开销。

主要用于**递归调用**，也可以用于分层或不规则任务，或在运行时根据数据特征决定后续并行任务。